TALLER DE DESARROLLO III · PLATAFORMA E-COMMERCE

Integración del Servicio de Notificaciones por Correo Electrónico

Proyecto: Markettaller
Tecnologías: Node.js, React, PostgreSQL y Nodemailer
Arquitectura: Puertos y Adaptadores
Fecha: 10 de octubre de 2026

Entrega técnica con descripción de la implementación, configuración de SMTP, evidencia de validación y anexo de código fuente.

Repositorio GitHub:
https://github.com/TitanLopezz/Markettaller

Rama de la actividad:
codex/email-notifications-delivery

1. Objetivo y alcance

Notificar al cliente y al administrador cuando se registra una compra. El cliente recibe el desglose, el estado Pendiente de Pago y, al elegir transferencia, las instrucciones bancarias. El administrador recibe una notificación de nueva orden con los datos de la compra.

La integración utiliza el checkout de clientes existente. El caso createOrder.js de solicitudes de proveedores conserva su responsabilidad independiente. Los precios, los totales y el correo del cliente se obtienen del servidor, no del cuerpo de la solicitud del navegador.

2. Diseño hexagonal

Dominio: puerto de salida

EmailServicePort define sendOrderConfirmationEmail(orderData) y sendAdminNewOrderNotification(orderData). No depende de SMTP, Nodemailer, Express ni React.

Aplicación: caso de uso

CreateOrderUseCase recibe emailService y logger por inyección. Su método run ejecuta la persistencia proporcionada por saveOrder, espera el commit y coordina ambos envíos.

Infraestructura: adaptador

NodemailAdapter extiende el puerto y usa un transporte SMTP inyectado. Las plantillas generan HTML y texto plano y escapan los valores incluidos en HTML.

Bootstrap: composición

createCommerce.js construye el transporte de Nodemailer, el adaptador y los casos de uso. La configuración de entorno se lee únicamente en las capas externas.

Presentación: React

Storefront invoca store.checkout; el servicio de aplicación usa ShopGateway para realizar POST /api/shop/checkout con autenticación y clave idempotente.

```
React -> store.checkout -> ShopGateway -> POST /api/shop/checkout
  -> checkout / CreateOrderUseCase.run
  -> unidad de trabajo + repositorio -> COMMIT
  -> EmailServicePort -> NodemailAdapter -> servidor SMTP
```

3. Secuencia y tratamiento de errores

1. El backend valida cliente, dirección, cantidades y método de pago.
2. La transacción valida existencias, calcula precios y envío, guarda el pedido y descuenta stock.
3. Después del commit, se consulta el pedido persistido y se prepara el destinatario.
4. Promise.allSettled permite intentar los dos correos de forma independiente.
5. La respuesta agrega email_notification y React muestra el comprobante.

Un fallo SMTP no revierte una compra guardada. accepted significa que el servicio SMTP aceptó el mensaje, no que llegó a la bandeja del usuario. unavailable mantiene el comprobante y las instrucciones visibles. Un reintento con la misma clave idempotente no vuelve a enviar los correos.

4. Configuración para Mailtrap o Ethereal

Instala las dependencias del backend y configura backend/.env con tus credenciales SMTP de prueba. No agregues este archivo a GitHub. El archivo .env.example solo contiene nombres y valores de ejemplo.

```
npm install --prefix backend

SMTP_HOST=smtp.ethereal.email
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=usuario_de_prueba
SMTP_PASSWORD=credencial_de_prueba
MAIL_FROM=Tienda <tienda@example.test>
ADMIN_NOTIFICATION_EMAIL=admin@example.test
TRANSFER_INSTRUCTIONS="Banco: banco real; Beneficiario: titular; CLABE: cuenta real"
```

Para Mailtrap Sandbox, sustituye host, puerto, usuario y contraseña por los de tu sandbox. Para Ethereal, usa las credenciales de una cuenta de prueba. Ambos servicios permiten inspeccionar los mensajes de prueba en su bandeja web. En producción debe configurarse un servicio de entrega real.

TRANSFER_INSTRUCTIONS habilita la transferencia y contiene banco, beneficiario y CLABE/cuenta de la tienda. Cada mensaje añade el número del pedido como referencia. Se conserva el método de pago contra entrega y la verificación manual de las transferencias.

5. Validación realizada

Backend: 31 pruebas aprobadas; 2 pruebas de integración PostgreSQL omitidas.
Frontend: 8 pruebas aprobadas.
React: compilación de producción y lint correctos.

Las pruebas nuevas verifican plantillas, importes persistidos, escape de HTML, los dos destinatarios, aislamiento de fallos SMTP, ausencia de falsas confirmaciones sin configuración, ejecución después del commit y omisión de envíos en solicitudes repetidas.

```
npm test --prefix backend
npm test --prefix frontend
npm run build --prefix frontend
npm run lint --prefix frontend
```

El envío real no se ha validado: requiere las credenciales SMTP del entorno. Para la comprobación manual, crear una compra con transferencia, revisar ambos mensajes en Mailtrap/Ethereal y verificar desglose, referencia e instrucciones bancarias.

6. Límites y consideraciones

No se implementó una cola persistente ni reintentos durables. Una interrupción del proceso después del commit puede perder la notificación. Para entrega eventual se requiere un outbox transaccional con un worker. La recuperación de contraseñas y los cambios posteriores de estado conservan el servicio Resend existente.

Durante la instalación npm reportó tres vulnerabilidades de severidad alta en el árbol de dependencias. No se aplicaron actualizaciones incompatibles como parte de esta actividad.

7. Código fuente en GitHub

El anexo reproduce el código principal de la actividad. La implementación integrable, el archivo de dependencias y las pruebas se encuentran en la rama de la actividad en GitHub.

Anexo de código

backend/src/domain/ports/EmailServicePort.js

```
/** Puerto de salida: datos del pedido persistido, nunca precios del navegador. */
class EmailServicePort {
  async sendOrderConfirmationEmail(orderData) { throw new Error('Método no implementado.'); }
  async sendAdminNewOrderNotification(orderData) { throw new Error('Método no implementado.'); }
}
module.exports = { EmailServicePort };
```

Anexo de código

backend/src/infrastructure/adapters/NodemailAdapter.js

```
const { EmailServicePort } = require('../../domain/ports/EmailServicePort');
const { orderEmailTemplate } = require('./orderEmailTemplates');

/** SMTP funciona tanto con Mailtrap Sandbox como con una cuenta Ethereal. */
class NodemailAdapter extends EmailServicePort {
  constructor({transport, from, adminEmail}) {
    super();
    this.transport = transport;
    this.from = from;
    this.adminEmail = adminEmail;
  }
  async deliver(to, order, admin) {
    if (!this.transport || !this.from || !to) return false;
    const result = await this.transport.sendMail({from:this.from, to, ...orderEmailTemplate(order,
    admin)});
    return Boolean(result.accepted?.length) && !result.rejected?.length;
  }
  sendOrderConfirmationEmail(order) { return this.deliver(order.customer.email, order, false); }
  sendAdminNewOrderNotification(order) { return this.deliver(this.adminEmail, order, true); }
}
module.exports = { NodemailAdapter };
```

Anexo de código

backend/src/infrastructure/adapters/orderEmailTemplates.js

```
const escapeHtml = value => String(value ?? '').replace(/[&<>"']/g, char =>
    ({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;","'":'&#39;'}[char]));
const money = cents => new Intl.NumberFormat('es-MX', {style:'currency',
    currency:'MXN'}).format(Number(cents) / 100);

function orderEmailTemplate(order, admin = false) {
  const subject = `${admin ? 'Nueva orden recibida' : 'Confirmación de compra'} #${order.id}`;
  const lines = order.items.map(item => `${item.product_name} · ${item.quantity} ×
    ${money(item.unit_price_cents)} = ${money(item.quantity * Number(item.unit_price_cents))}`);
  const payment = order.payment_method === 'transferencia'
    ? `Transferencia bancaria:\n${order.transfer_instructions}\nReferencia: pedido #${order.id}. El
    pago se verificará manualmente.`
    : 'Forma de pago: contra entrega.';
  const paragraphs = [subject, admin ? `Cliente: ${order.customer.email}` : `Hola,
    ${order.customer.name || 'cliente'}. Recibimos tu compra.`,
    'Estado del pago: Pendiente de Pago', ...lines,
    `Subtotal: ${money(order.subtotal_cents)}`, `Envío: ${money(order.shipping_cents)}`, `Total:
    ${money(order.total_cents)}`, payment];
  return {subject, text:paragraphs.join('\n\n'), html:`<!doctype html><html
    lang="es"><body><h1>${escapeHtml(subject)}</h1>${paragraphs.slice(1).map(p =>
    `<p>${escapeHtml(p).replace(/\n/g, '<br>')}</p>`).join('')}</body></html>`};
}
module.exports = { orderEmailTemplate };
```

Anexo de código

backend/src/application/use-cases/CreateOrderUseCase.js

```
/** Ejecutar únicamente después del commit. Un fallo SMTP no revierte la compra. */
class CreateOrderUseCase {
  constructor({emailService, logger}) {
    this.emailService = emailService;
    this.logger = logger;
  }
  async run({saveOrder, loadCustomer, transferInstructions}) {
    // saveOrder coordina repositorio y unidad de trabajo; resuelve tras el commit.
    const {order, repeated} = await saveOrder();
    if (!repeated) {
      try {
        const customer = await loadCustomer(order.user_id);
        order.email_notification = await this.notify({...order, customer,
    transfer_instructions:transferInstructions});
      } catch {
        this.logger.error('No se pudo preparar el correo del pedido.');
        order.email_notification = {customer:'unavailable', admin:'unavailable'};
      }
    }
    return order;
  }
  async notify(orderData) {
    const results = await Promise.allSettled([
      Promise.resolve().then(() => this.emailService.sendOrderConfirmationEmail(orderData)),
      Promise.resolve().then(() => this.emailService.sendAdminNewOrderNotification(orderData)),
    ]);
    if (results.some(result => result.status === 'rejected')) {
      this.logger.error('Falló una notificación por correo del pedido.');
    }
    const status = result => result.status === 'fulfilled' && result.value === true ? 'accepted' :
    'unavailable';
    return {customer:status(results[0]), admin:status(results[1])};
  }
}
module.exports = { CreateOrderUseCase };
```

Anexo: integración con el checkout

backend/src/application/use-cases/commerce.js (extractos)

```
const {fail}=require('../../domain/errors');
const {text,positiveId,addressInput,checkoutInput,toCents}=require('../../domain/commerceInput');
const {orderChanges,assertFulfillmentChanges}=require('../../domain/shopOrder');
const {CreateOrderUseCase}=require('./CreateOrderUseCase');

const createCommerceUseCases=({repository,unitOfWork,passwordHasher,crypto,mailer,emailService,clock,lo
    gger,config,frontendOrigin})=>{
  const createOrderUseCase=emailService?new CreateOrderUseCase({emailService,logger}):null;
  const getOrder=async(id,userId,staff=false)=>{
    const order=await repository.findOrder(positiveId(id));
    if(!order||(!staff&&Number(order.user_id)!==Number(userId)))throw fail('Pedido no
    encontrado.','NOT_FOUND');

    return {order:await getOrder(result.id,userId),repeated:Boolean(result.repeated)};
  };
  const checkout=async(userId,input)=>{
    if(createOrderUseCase)return createOrderUseCase.run({saveOrder:()=>saveOrder(userId,input),loadCust
    omer:id=>repository.findUser(id),transferInstructions:config.transfer_instructions});
    const {order,repeated}=await saveOrder(userId,input);
    if(!repeated)await mailOrder(order);
    return order;
  };
```

saveOrder conserva el flujo transaccional existente. Las instrucciones de pago se agregan a los datos destinados al puerto; no se obtienen de la solicitud del cliente.

frontend/src/presentation/Storefront.jsx (extractos)

```
  const checkout = async event => {
    event.preventDefault();setError('');setBusy(true)
    try {
      const data = {items:cart,address,payment_method:payment}
      const order=await store.checkout(data,{token})
      setReceipt(order);setCart([]);setView('receipt');setReload(v=>v+1)
    }catch(e){setError(e.message)}finally{setBusy(false)}
  }
```

```
<p role="status">
  {receipt.email_notification?.customer === 'accepted'
    ? 'La confirmación se envió al servicio de correo. Revisa tu bandeja de entrada y spam; incluye el
    desglose y las instrucciones de pago.'
    : 'Tu pedido quedó guardado. Consulta el comprobante y las instrucciones de pago aquí aunque el
    correo no esté disponible.'}
</p>
```

El componente muestra OrderReceipt y, para transferencia, config.transfer_instructions y la referencia del pedido. El botón de compra queda deshabilitado durante el envío.

Anexo de código

backend/src/bootstrap/createCommerce.js

```
const {createCommerceUseCases}=require('../application/use-cases/commerce');
const
    {PostgresCommerceRepository}=require('../infrastructure/repositories/PostgresCommerceRepository');
const {PostgresUnitOfWork}=require('../infrastructure/database/PostgresUnitOfWork');
const {BcryptPasswordHasher}=require('../infrastructure/services/BcryptPasswordHasher');
const {NodeCryptoService}=require('../infrastructure/services/NodeCryptoService');
const {ResendMailer}=require('../infrastructure/services/ResendMailer');
const {commerceConfig}=require('../infrastructure/config/commerceConfig');
const {assertPort}=require('../application/ports/contracts');
const {NodemailAdapter}=require('../infrastructure/adapters/NodemailAdapter');
const createCommerce=(pool,env=process.env,mailOverride)=>{
  const transport=env.SMTP_HOST?require('nodemailer').createTransport({host:env.SMTP_HOST,port:Number(e
    nv.SMTP_PORT||587),secure:env.SMTP_SECURE==='true',auth:{user:env.SMTP_USER,pass:env.SMTP_PASSWORD}
    ,connectionTimeout:10000,greetingTimeout:10000,socketTimeout:15000}):null;
  const emailService=new
    NodemailAdapter({transport,from:env.MAIL_FROM,adminEmail:env.ADMIN_NOTIFICATION_EMAIL});
  const config=commerceConfig(env);if(mailOverride)config.password_recovery_enabled=true;
  return createCommerceUseCases({emailService,repository:assertPort('CommerceRepository',new
    PostgresCommerceRepository(pool)),unitOfWork:assertPort('UnitOfWork',new
    PostgresUnitOfWork(pool)),passwordHasher:assertPort('PasswordHasher',new
    BcryptPasswordHasher()),crypto:assertPort('CryptoService',new
    NodeCryptoService()),mailer:assertPort('Mailer',mailOverride?{send:mailOverride}:new
    ResendMailer({apiKey:env.RESEND_API_KEY,from:env.MAIL_FROM})),clock:{now:()=>new
    Date()},logger:console,config,frontendOrigin:env.FRONTEND_ORIGIN});
};
module.exports={createCommerce};
```

Anexo de código

backend/test/orderEmail.test.js

```
const test = require('node:test');
const assert = require('node:assert/strict');
const {NodemailAdapter} = require('../src/infrastructure/adapters/NodemailAdapter');
const {CreateOrderUseCase} = require('../src/application/use-cases/CreateOrderUseCase');
const order = {id:7,customer:{name:'Ana <script>',email:'ana@example.test'},items:[{product_name:'Libro
     & guía',quantity:2,unit_price_cents:15000}],subtotal_cents:30000,shipping_cents:9900,total_cents:3
    9900,payment_method:'transferencia',transfer_instructions:'Banco de prueba\nCLABE: 000'};

test('SMTP recibe ambas plantillas, importes persistidos y HTML escapado', async () => {
  const messages=[];
  const adapter=new NodemailAdapter({transport:{async sendMail(message){messages.push(message);return
    {accepted:[message.to],rejected:[]};}},from:'tienda@example.test',adminEmail:'admin@example.test'})
    ;
  const useCase=new CreateOrderUseCase({emailService:adapter,logger:{error(){}}});
  assert.deepEqual(await useCase.notify(order),{customer:'accepted',admin:'accepted'});
  assert.equal(messages[0].to,order.customer.email);
  assert.equal(messages[1].to,'admin@example.test');
  assert.match(messages[0].text,/Pendiente de Pago/);
  assert.match(messages[0].text,/399/);
  assert.match(messages[0].text,/CLABE/);
  assert.match(messages[0].html,/Ana &lt;script&gt;/);
  assert.match(messages[0].html,/Libro &amp; guía/);
});
test('un fallo del cliente no impide la notificación administrativa', async () => {
  let adminSent=false;
  const useCase=new CreateOrderUseCase({emailService:{async sendOrderConfirmationEmail(){throw new
    Error('SMTP');},async sendAdminNewOrderNotification(){adminSent=true;return
    true;}},logger:{error(){}}});
  assert.deepEqual(await useCase.notify(order),{customer:'unavailable',admin:'accepted'});
  assert.equal(adminSent,true);
});
test('sin SMTP no se afirma un envío exitoso', async () => {
  const adapter=new NodemailAdapter({});
  assert.equal(await adapter.sendOrderConfirmationEmail(order),false);
});
test('envía después de guardar y omite correos para la clave repetida', async () => {
  const events=[];
  const useCase=new CreateOrderUseCase({emailService:{async
    sendOrderConfirmationEmail(){events.push('customer');return true;},async
    sendAdminNewOrderNotification(){events.push('admin');return true;}},logger:{error(){}}});
  const saveOrder=async()=>{events.push('commit');return {order:{...order,user_id:1},repeated:false};};
  await useCase.run({saveOrder,loadCustomer:async()=>order.customer,transferInstructions:order.transfer
    _instructions});
  assert.deepEqual(events,['commit','customer','admin']);
  events.length=0;
  await useCase.run({saveOrder:async()=>({order:{...order},repeated:true}),loadCustomer:async()=>{throw
     new Error('No debe consultarse');}});
  assert.deepEqual(events,[]);
});
```